

DESCRIPCIÓN ESTRUCTURA 1 ()

1. Estructura estandarizada (y ya verificada con OpenCode)

text

```
mi-proyecto/
├── .ai/
│   ├── system.md      # ← Prompt sistemático global (comportamiento, tono, reglas base)
│   ├── agents.md      # ← Definición de agentes especialistas y su rol
│   ├── rules.md       # ← Reglas OBLIGATORIAS y de seguridad del proyecto
│   ├── context.md     # ← Contexto técnico: stack, esquemas, paths, convenciones
│   └── memory.md      # ← (Opcional, pero recomendado) historial / patrones del proyecto
├── .env
├── .gitignore
├── opencode.json      # (si usas OpenCode: config de agente / model / paths)
└── src/ ...
```

Nota: No uses `systemprompt.md` — la documentación oficial de OpenCode y la mayoría de herramientas leen exclusivamente `system.md` dentro de `.ai/`.

2. Función y comportamiento de cada archivo



`.ai/system.md` — Prompt sistemático global

Función

Define *quién es el agente en este proyecto*, su tono, límites, estilo y reglas base. Es el “personaje” y el “contrato de comportamiento” que la IA no puede ignorar en ninguna sesión.

Comportamiento en el proyecto

- Se carga una sola vez al inicio de cada sesión del agente.

- Determina cómo la IA habla, piensa y limita sus acciones (qué puede / no puede tocar, qué formato usar, qué tono).
- Importa los otros archivos de `.ai/` (rules, context, agents) para completarse.
- Es el archivo más “fuerte”: si aquí dice “NUNCA toques `.env`”, la IA lo respeta en todas las interacciones.

Ejemplo de sección clave

Markdown

Comportamiento base

- Habla en español, tono técnico pero claro.
 - NUNCA modifiques archivos fuera de `/src` o `.ai`.
 - Cada sugerencia de código = (1) análisis, (2) código, (3) explicación.
-
- Usa las reglas de `@.ai/rules.md` como OBLIGATORIAS.



`.ai/agents.md` — Definición de agentes especialistas

Función

Define quién es cada agente, su rol, habilidades, tono y límites. Permite tener múltiples “personas” dentro del mismo proyecto (ej: *Arquitecto*, *DevJS*, *Tester*, *Docente*).

Comportamiento en el proyecto

- Cuando el usuario solicita algo específico (“ayúdame como el arquitecto”), la IA selecciona el agente correspondiente según esta definición.
- Cada agente hereda el comportamiento de `system.md`, pero sobrescribe o restringe su tono, herramientas y alcance.
- Evita que el agente “se salga de su rol” — cada uno sabe *exactamente* qué puede y no puede hacer.

Ejemplo

Markdown

Agente: Archito (Arquitecto)

- Rol: Define arquitectura, estructuras, patrones y decisiones de diseño.
- Tono: formal, enfocado en principio de diseño y mantenibilidad.
- Herramientas permitidas: diagramas, esbozos de arquitectura, consultas de BD.
- No genera código de implementación – solo diseño y especificación.

Agente: DevJS

- Rol: Implementación en JavaScript / TypeScript.
- Tono: práctico, conciso, con comentarios y tests mínimos.

- Puede modificar solo /src y archivos de tipo .js / .ts.



.ai/rules.md — Reglas OBLIGATORIAS del proyecto

Función

Contiene reglas de seguridad, calidad y proceso que no se pueden violar bajo ninguna circunstancia. Son *no-negociables*.

Comportamiento en el proyecto

- Se leen después de `system.md` (o como inclusión dentro de él) y se aplican en cada respuesta.
- La IA debe verificar cada acción o sugerencia contra estas reglas antes de responder.
- Si una sugerencia viola una regla → la IA debe rechazarla o pedir aclaración, no la ejecuta en silencio.

Ejemplo

Markdown

Reglas OBLIGATORIAS

1. NUNCA expongas valores de ``.env`` en respuestas, comentarios o archivos generados.
2. NUNCA elimines o renombra archivos de prueba (``.tests``, ``.spec``).
3. Cada modificación de base de datos requiere: (a) migración, (b) comentario de cambio, (c) test de retroceso.

4. El código debe pasar linter y test antes de ser mostrado como "listo".



`.ai/context.md` — Contexto técnico del proyecto

Función

Contiene todo lo que la IA necesita saber sobre el stack, estructura, esquemas, convenciones y paths — sin tener que leer el código entero.

Comportamiento en el proyecto

- Se inyecta en la memoria de la IA al inicio de cada sesión → la IA sabe automáticamente:
 - qué librerías usa,
 - estructura de carpetas,
 - esquemas de BD,
 - nombres de variables / constantes clave,
 - convenciones de nomenclatura.
- Evita que la IA “adivine” o invente nombres — todo está documentado y fijo.

Ejemplo

Markdown

Stack

- Language: TypeScript + Node 20
- Framework: Express 4.x
- DB: PostgreSQL + Prisma
- Test: Jest + Supertest

Estructura clave

```
/src
  /api      → rutas y controladores
  /db       → modelos Prisma y queries
  /services → lógica de negocio
  /utils    → helpers reutilizables
```

Convenciones

- Nombres de archivos: ``camelCase.ts``
- Nombres de tablas BD: ``snake_case``

- Todos los endpoints devuelven `{ success, data, error }`



.ai/memory.md — (Recomendado) Historial / patrones

del proyecto

Función

Almacena decisiones ya tomadas, patrones probados, errores frecuentes y decisiones de diseño pasadas. Es el “conciencia acumulada” del proyecto.

Comportamiento en el proyecto

- La IA lo consulta en cada sesión para no repetir errores o decisiones obsoletas.
- Es útil para proyectos maduros: evita que la IA proponga soluciones ya probadas como “no funcionales” o obsoletas.
- Se actualiza manualmente (o automáticamente si tu herramienta lo soporta) tras cada decisión importante.

Ejemplo

Markdown

Decisiones ya tomadas

- No usamos Redux; estado se maneja por contexto + React Query.
- No usamos migraciones manuales de SQL; solo Prisma.
- Todos los logs se envían a Datadog — no a consola.

Errores a evitar

- No uses `any` en tipos de Prisma — causa bugs en migraciones.

- No modifiques `schema.prisma` directamente; usa `prisma generate` después de ediciones.

Este archivo no es obligatorio, pero es el que más valor aporta a proyectos grandes o de larga vida.

3. Ejemplo completo y funcional de proyecto

Supongamos un proyecto “e-commerce-api” (Node + TS + Prisma + PostgreSQL). Aquí está la carpeta `.ai/` lista para usar con OpenCode:

text

```
e-commerce-api/
├── .ai/
│   ├── system.md
│   ├── agents.md
│   ├── rules.md
│   ├── context.md
│   └── memory.md
├── .env
├── .gitignore
├── opencode.json
├── src/
│   ├── api/
│   ├── db/
│   └── services/
└── ...
```

`.ai/system.md`

Markdown

System Prompt – e-commerce-api

Identidad

Eres el asistente de desarrollo de ****e-commerce-api****. Tu objetivo es ayudar solo dentro de este repositorio, respetando estrictamente las reglas y contexto de ``.ai/``.

Reglas base (OBLIGATORIAS)

@.ai/rules.md

Agentes disponibles

@.ai/agents.md

Contexto técnico

@.ai/context.md

Comportamiento

- Habla en español, tono técnico y claro.
- Cada sugerencia de código = (1) análisis, (2) código, (3) explicación corta.

- NUNCA toques ``.env``, archivos de despliegue o carpetas de pruebas sin autorización explícita.
- Si usas un agente especialista, comienza la respuesta con su nombre.

`.ai/agents.md`

Markdown

Agente: Archito (Arquitecto)

- Rol: Define arquitectura, estructuras, APIs y decisiones de diseño.
- Tono: formal, enfocado en mantenibilidad y escalabilidad.
- No genera código de implementación – solo diseño y especificación.
- Herramientas: diagramas, esbozos de API, decisiones de arquitectura.

Agente: DevTS (Desarrollador TS)

- Rol: Implementación en TypeScript / Node.
- Tono: práctico, conciso, con comentarios y tests mínimos.
- Puede modificar solo ``.src`` y archivos ``.ts``.
- Genera tests con Jest cuando sea necesario.

Agente: Tester

- Rol: Diseña y ejecuta tests, verifica cobertura y comportamiento.
- Tono: metódico, enfocado en casos límite y retroceso.
- Modifica solo ``.tests`` y archivos de prueba.

`.ai/rules.md`

Markdown

Reglas OBLIGATORIAS – e-commerce-api

1. NUNCA expongas claves de DB, API o ``.env`` en respuestas o archivos generados.
2. NUNCA elimines o renombres archivos de prueba (``.tests``, ``.spec``).
3. Cada cambio en la base de datos requiere:
 - migración de Prisma,
 - comentario de cambio en ``.MIGRATION_NOTES.md``,
 - test de retroceso.
4. El código debe pasar ``.pnpm lint`` y ``.pnpm test`` antes de ser marcado como “listo”.
5. No uses ``.any`` en tipos de Prisma – causa errores en migraciones.
6. Todos los endpoints devuelven el formato: ``. { success: bool, data?: any, error?: string }``.

`.ai/context.md`

Markdown

Stack

- Language: TypeScript (Node 20)
- Framework: Express 4.x

- DB: PostgreSQL + Prisma
- Test: Jest + Supertest
- Build / lint: pnpm, eslint, prettier

Estructura clave

```
/src
  /api      → rutas y controladores
  /db       → modelos Prisma + queries
  /services → lógica de negocio (carrito, pagos, usuarios)
  /utils    → helpers reutilizables

/db
  /prisma   → schema y migraciones

/tests
  /api      → tests de endpoints
  /db       → tests de queries
```

Convenciones

- Nombres de archivos: ``camelCase.ts``
- Tablas BD: ``snake_case``
- Todos los endpoints devuelven ``{ success, data, error }``
- No se usan promesas sin ``async/await`` – siempre con try/catch

`.ai/memory.md`

Markdown

Decisiones ya tomadas

- Estado de sesión se maneja con React Query, no Redux.
- Pagos se integran vía Stripe – no se almacenan tokens en DB.
- Migraciones de Prisma se ejecutan con ``pnpm prisma migrate dev``.
- Logs se envían a Datadog – no a consola.

Errores a evitar

- No modificar ``schema.prisma`` directamente; siempre por migración.
- No usar ``console.log`` en código de producción.
- No omitir validación de input en endpoints de usuario (usar Zod).

4. ¿Faltan archivos?

Para un proyecto completo, profesional y totalmente autónomo bajo un agente IA, la lista completa y estandarizada es la siguiente:

Archivo	Obligatorio?	Función
<code>.ai/system.md</code>	✓ Sí	Prompt sistemático global (comportamiento, tono, límites)
<code>.ai/agents.md</code>	✓ Sí	Definición de agentes especialistas
<code>.ai/rules.md</code>	✓ Sí	Reglas OBLIGATORIAS y de seguridad
<code>.ai/context.md</code>	✓ Sí	Stack, estructura, convenciones técnicas
<code>.ai/memory.md</code>	⚙ Recomendado	Historial de decisiones y patrones del proyecto
<code>opencode.json</code>	⚙ Opcional	Configuración específica de OpenCode (model, paths, agentes predefinidos)
<code>.env</code>	✓ Sí	Keys y variables de entorno (nunca a git)